

DESIGN AND ANALYSIS OF ALGORITHMS

MODEL PRACTICAL EXAM.

1) Find Maximum Element in array.

```
arr = list(map(int, input("Enter element: ").split()))
```

```
maximum = arr[0]
```

```
for i in arr:
```

```
    if i > maximum:
        maximum = i
```

```
print("Maximum element:", maximum)
```

sample input:

10 25 5 40 15

Output

Maximum element: 40

2) Find Peak element in array using $O(\log n)$

```
def find_peak(arr):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low < high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] < arr[mid + 1]:
```

```
            low = mid + 1
```

```
    else:
```

```
        high = mid
```

```
    return arr[low]
```



```

arr = List (map (int, Input ("Enter elements ") . split ' '))
Print ("Peak element : ", findPeak (arr))

```

Sample Input

1 5 20 41 0

Output

Peak element is 20

5) MEDIAN OF MEDIAN'S ALGORITHM TO FIND K-th Smallest Element.

```

def median_of_medians(arr, k):

```

```

    if len(arr) <= 5:

```

```

        return sorted(arr)[k]

```

```

    group = [arr[i:i+5] for i in range(0, len(arr), 5)]

```

```

    medians = [sorted(group)[len(group)//2] for group in
                group]

```

```

    pivot = median_of_medians(medians, len(medians)//2)

```

```

    low = [x for x in arr if x < pivot]

```

```

    high = [x for x in arr if x > pivot]

```

```

    equal = [x for x in arr if x == pivot]

```

```

    if k < len(low):

```

```

        return median_of_medians(low, k)

```

```

    elif k < len(low) + len(equal):

```

```

        return pivot

```


else:

return median of median (high, k - len(low))

sample input

12 5 5 7 4 19

Output

3th smallest element : 5

4) NUMBER OF UNIQUE PATHS IN AN $m \times n$ GRID
using Dynamic Programming.

```
m = int(input("enter rows (m) : "))
```

```
n = int(input("enter columns (n) : "))
```

```
dp = [[0 for j in range(n)] for i in range(m)]
```

```
for i in range(1, m):
```

```
    for j in range(1, n):
```

```
        dp[i][j] = dp[i-1][j] + dp[i][j-1]
```

```
print("Number of unique paths : ", dp[m-1][n-1])
```


Sample Input

2

3

Output

Number of unique paths